

Enterprise RAG

A Retrieval-Augmented Generation system for answering employee questions from HR & company policy documents — with hybrid retrieval, citation-backed answers, and hallucination guardrails.

Repository: github.com/sai-teja33/enterprise-rag

Prepared for: Manager review / project walkthrough

Prepared by: Sai Teja

Document date: July 30, 2026

Table of Contents

- 1. Executive Summary
- 2. What Problem This Solves
- 3. System Architecture
- 4. Technology Stack
- 5. Document Ingestion Pipeline
- 6. Retrieval Engine (Hybrid Search + Reranking)
- 7. Answer Generation & Guardrails
- 8. API Reference
- 9. Data Model
- 10. Frontend Application
- 11. Evaluation Framework
- 12. Current Status & Known Gaps
- 13. Suggested Next Steps
- 14. Appendix — Full Dependency List

1. Executive Summary

Enterprise RAG is a full-stack Retrieval-Augmented Generation application that lets employees ask natural-language questions about company HR policies (leave, insurance, code of conduct, onboarding, payroll, etc.) and get grounded, citation-backed answers — instead of manually searching through PDF policy documents.

The project has three parts:

- A **FastAPI backend** that ingests documents, classifies them, chunks and embeds them, and answers questions using a hybrid retrieval + LLM pipeline.
- An **Angular 21 frontend** with a dashboard, a document management screen, and a chat-style "query playground" for asking questions and inspecting retrieval evidence.
- A lightweight **evaluation harness** (36 test questions) that measures answer accuracy, citation correctness, and abstention behavior automatically.

Headline design decision: the system is built to *refuse to answer* rather than hallucinate. Every answer must be grounded in retrieved document chunks, scored against two independent confidence checks, and returned with the exact chunk/page it came from — or the system explicitly says it couldn't find a reliable answer.

This is a working prototype (not yet in production) with a functioning end-to-end pipeline, a deployed frontend, and a real evaluation suite. Section 12 covers exactly what is complete versus still in progress.

2. What Problem This Solves

In most companies, HR and policy information is scattered across long PDF documents (employee handbooks, leave policies, insurance documents, IT security policies, etc.). Employees either:

- Ping HR directly with repetitive questions ("How many sick leaves do I get?", "Does insurance cover my parents?"), or
- Skim multi-page PDFs themselves and often misread nuanced eligibility rules.

Enterprise RAG addresses this by turning the company's own policy documents into a searchable knowledge base that can answer specific questions **with a citation to the exact source page**, and — critically — say "I don't know" when the documents don't support an answer, rather than guessing.

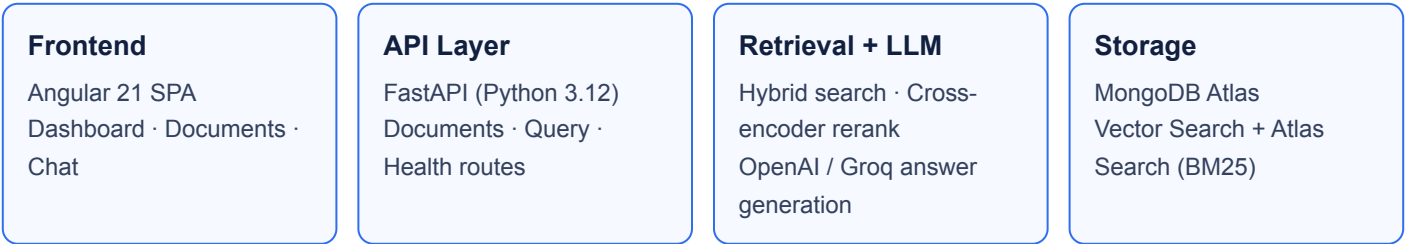
Sample document set included in the repo

The project ships with 13 representative HR PDF documents used for development and testing, including:

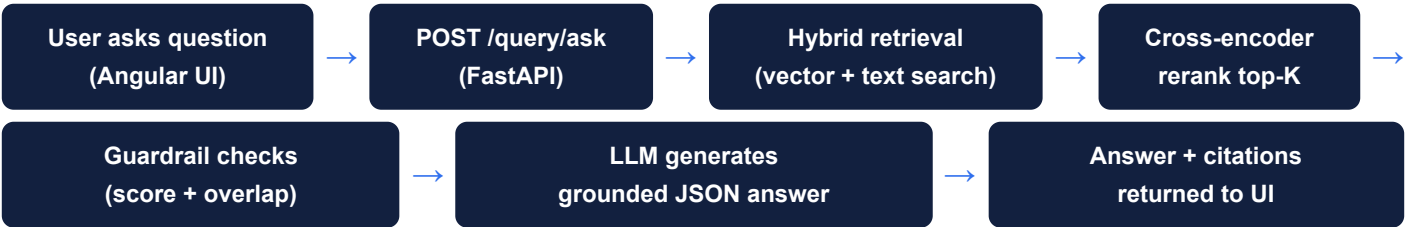
- Employee Handbook
- Employee Leave Policy
- Medical Insurance Policy
- Code of Conduct Policy
- Onboarding & Offboarding Policy
- Performance Management Policy
- Password / MFA Policy
- Software Installation Policy

3. System Architecture

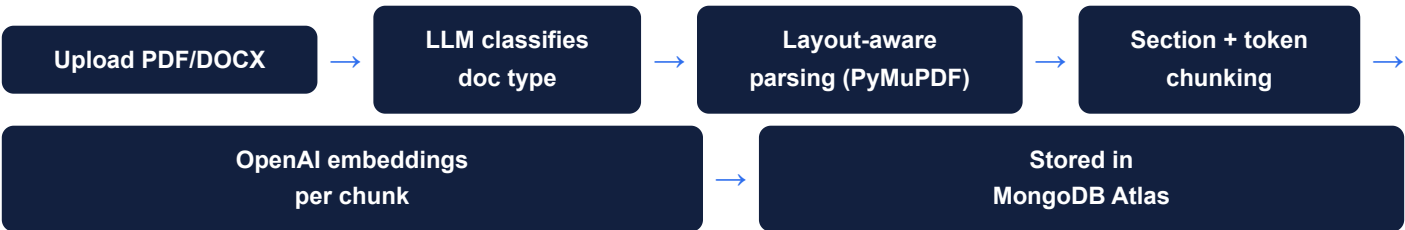
At a high level, the system has four layers: ingestion, storage, retrieval, and presentation.



End-to-end request flow — asking a question



End-to-end flow — ingesting a document



4. Technology Stack

Layer	Technology	Purpose
Backend framework	FastAPI (Python 3.12.4), Uvicorn	REST API server
Database	MongoDB Atlas (via PyMongo)	Document + chunk storage, Vector Search, Atlas Search
Embeddings	OpenAI <code>text-embedding-3-small</code>	Turns chunk text into vectors for semantic search
Reranker	Sentence-Transformers Cross-Encoder <code>ms-marco-MiniLM-L-12-v2</code>	Locally-run relevance reranking of candidate chunks
Answer LLM	OpenAI (<code>gpt-4o-mini</code>) or Groq (<code>llama-3.3-70b-versatile</code>)	Swappable via config — generates the final grounded answer
Document classification LLM	OpenAI (<code>gpt-4o-mini</code>) via LangChain	Auto-labels uploaded documents into HR policy categories
PDF parsing	PyMuPDF (fitz), pypdf	Layout-aware text + font/bold extraction for heading detection
DOCX parsing	docx2txt	Plain text extraction from Word documents
Tokenization	tiktoken	Token-accurate chunk sizing for the embedding model
Orchestration helpers	LangChain (core, openai, text-splitters)	Prompting, recursive character text splitting
Frontend framework	Angular 21 (standalone components)	Single-page application
UI components	Angular Material, AG Grid	Tables, dialogs, forms, data grids
Frontend hosting	Vercel	Live demo deployment of the Angular client
Evaluation	Python + <code>requests</code> , JSON/CSV reports	Automated answer-quality scoring

5. Document Ingestion Pipeline

This is the process that turns a raw PDF or DOCX file into searchable, embedded chunks. It happens either one document at a time (upload flow) or in bulk (`/documents/bulk-ingest`), which scans the `server/data/raw` folder).

Step 1 — Upload & Classification

When a file is uploaded, the first two pages are extracted as a text preview, and an LLM call (GPT-4o-mini via LangChain) classifies the document into one of **20 predefined HR categories** — e.g. `leave_policy`, `insurance_policy`, `code_of_conduct`, `password_mfa_policy`, `onboarding_policy`, falling back to `unknown` if nothing matches confidently.

Step 2 — Layout-Aware Parsing

PDFs are parsed page-by-page using PyMuPDF at the text-block level, capturing for every block: the text, its **font size**, whether it's **bold**, and whether it looks like a table or a list. This metadata is what powers heading detection later — it's not just plain text extraction. DOCX files use a simpler plain-text extraction (docx2txt), without layout metadata.

Step 3 — Section-Aware Chunking

Rather than naively splitting every N characters, the system first reconstructs the document's logical structure:

- **Heading detection** — combines font-size/bold signals with heuristics (numbered headings like "2.1", ALL-CAPS titles, short title-case lines) while explicitly excluding lines that look like body text, questions, or document metadata (e.g. "Version", "Effective Date").
- **Section splitting** — text is split into sections under each detected heading, preserving parent/child heading hierarchy (e.g. "2.1 Sick Leave" under "2. Leave Policy").
- **Small-section merging** — sections under ~700 characters are merged with neighbors so chunks aren't too thin to be useful.
- **Token-bounded splitting** — each section is further split with a tiktoken-aware recursive character splitter at **500 tokens per chunk with 75-token overlap**, so no chunk exceeds the embedding model's effective context.

If section detection produces nothing usable (e.g. a document with no discernible structure), the pipeline falls back to plain recursive token chunking of the full text.

Step 4 — Embedding

Each chunk is embedded with OpenAI `text-embedding-3-small`, but not just on its raw text — the embedded string is enriched with a short synthetic header first:

```
Document: {title}
Document Type: {doc_type}
Section: {section_title}
Pages: {page_range}

{chunk_text}
```

This gives the embedding model extra context (which document/section a chunk belongs to) so semantically similar chunks from different policies are less likely to be confused with each other.

Step 5 — Storage

Chunks are written to MongoDB with their text, embedding vector, token count, section metadata, and page range, ready for retrieval.

6. Retrieval Engine (Hybrid Search + Reranking)

This is the core of the system's answer quality. Every question runs through three independent retrieval/ranking stages before an LLM ever sees the data.

Stage 1 — Dense (Vector) Retrieval

The question is embedded with the same OpenAI embedding model, then matched against stored chunk vectors using a MongoDB Atlas `$vectorSearch` aggregation (top 10 candidates by default, `numCandidates: 50`).

Stage 2 — Sparse (Text/Keyword) Retrieval

In parallel, a MongoDB Atlas `$search` full-text query runs across the chunk text and title fields (top 10 candidates by default) — this catches exact terminology matches (e.g. specific policy terms) that a pure vector search can sometimes miss.

Stage 3 — Merge via Reciprocal Rank Fusion (RRF)

Results from both retrievers are merged into one deduplicated candidate list. Each chunk's rank in each retriever contributes $1 / (60 + \text{rank})$ to a combined RRF score, so chunks that rank well in *either* method — and especially chunks that appear in *both* — rise to the top.

Stage 4 — Cross-Encoder Reranking

The merged candidate pool is then reranked with a locally-run cross-encoder model (`ms-marco-MiniLM-L-12-v2`), which scores each (question, chunk) pair jointly — a more accurate but more expensive method than embedding similarity alone. Only the top 5 chunks (configurable) survive to the answer stage.

Why hybrid + rerank instead of vector search alone? Vector search is great at semantic similarity but can miss exact keyword matches (e.g. specific numbers, policy codes); keyword search is great at exact terms but misses paraphrased questions. Combining both and then reranking gets the benefits of each while filtering out the noise either one introduces alone.

7. Answer Generation & Guardrails

Once the top chunks are selected, the system decides whether it's even safe to attempt an answer before calling the LLM — this is the anti-hallucination layer.

Guardrail checks (before calling the LLM)

Check	Logic	Outcome if failed
Minimum vector confidence	Top vector similarity score < 0.65 and lexical overlap < 0.34	Abstain immediately, no LLM call
Lexical overlap check	Zero question keywords found anywhere in retrieved chunks	Abstain immediately, no LLM call

Lexical overlap is computed by stripping stopwords/common HR terms from the question, then measuring what fraction of the remaining meaningful words actually appear in the retrieved chunk text.

Structured LLM answer with four answer modes

When the guardrails pass, the LLM (OpenAI or Groq, selected by config) is prompted to return **strict JSON** with one of four answer modes:

Mode	When used
<code>direct_answer</code>	Context clearly supports one single answer.
<code>scoped_answer</code>	The question is broad or the policy varies by employee type / category / condition — the model must not collapse these into one universal answer.
<code>partial_answer</code>	Only part of the answer is supported by the retrieved chunks, but that part is still useful.
<code>not_found</code>	Insufficient evidence — returns a fixed abstention message, no citations attached.

The model is explicitly instructed **not to use outside knowledge** and not to invent numbers, timelines, or eligibility rules — only to work from the supplied chunks, and to cite exactly which chunk numbers support its answer.

The "rescue pass"

If the first LLM pass abstains (`not_found`) but retrieval confidence was actually quite high (vector score ≥ 0.80 **and** either rerank score ≥ 1.0 or lexical overlap ≥ 0.60), the system runs a **second LLM pass** with a prompt nudging it to be less conservative — since strong retrieval evidence combined with an over-cautious abstention is treated as a likely false negative. This rescue is only attempted once, and only when retrieval signals justify it.

Citations

Every non-abstained answer returns structured citation objects — document title, doc type, file name, page range, section title, and the retrieval/rerank scores that got that chunk selected — so the answer is fully traceable back to a specific page in a specific policy document.

8. API Reference

Documents (`/documents`)

Method & Path	Purpose
POST <code>/documents/upload</code>	Upload a PDF/DOCX, auto-classify it, and store its metadata
GET <code>/documents/documents</code>	List all uploaded documents
GET <code>/documents/{id}/preview</code>	Return the first ~3000 characters of extracted text
POST <code>/documents/{id}/chunk</code>	Run chunking for a document
GET <code>/documents/{id}/chunks</code>	List chunks with previews and metadata
POST <code>/documents/{id}/embed</code>	Generate embeddings for any un-embedded chunks
POST <code>/documents/{id}/reprocess</code>	Delete existing chunks and re-run chunk + embed in one call
POST <code>/documents/bulk-ingest</code>	Process every file in <code>server/data/raw</code> (skips already-processed files unless <code>reprocess_existing=true</code>)
GET <code>/documents/status</code>	Per-document chunk/embedding counts and "ready for query" flag

Query (`/query`)

Method & Path	Purpose
POST <code>/query/search</code>	Return raw hybrid+reranked chunks for a question (no LLM answer — useful for debugging retrieval quality)
POST <code>/query/ask</code>	Full pipeline: retrieval → guardrails → LLM answer → citations. Accepts <code>question</code> , <code>top_k</code> , and a <code>debug</code> flag for verbose retrieval diagnostics.

Health

Method & Path	Purpose
GET <code>/</code>	Root status message
GET <code>/health</code>	API + MongoDB connectivity check

9. Data Model

Two MongoDB collections back the whole system:

documents

Field	Description
title, file_name, file_path	Identifying info for the source file
doc_type	LLM-assigned HR category
uploaded_at, processed_at	Timestamps
chunking_strategy, chunk_count	Processing metadata, set after chunking runs

document_chunks

Field	Description
chunk_text, embedding_text	Raw chunk text and the context-enriched version actually embedded
embedding	OpenAI vector (only present once embedding has run)
page_start, page_end, section_title, parent_section, heading_level	Structural/positional metadata used for citations
chunk_index, chunk_size, token_count, chunking_strategy	Processing metadata

Two MongoDB Atlas search indexes back retrieval: `chunk_vector_index` (vector search on the `embedding` field) and `chunk_text_index` (full-text search on `chunk_text` / `title`).

10. Frontend Application

Built with Angular 21 using standalone components, Angular Material for UI, and AG Grid for tabular data. Three main screens, all inside a shared app shell:

Dashboard

Shows document processing health at a glance: total documents, how many are fully embedded and "ready for query," and total chunk count across the corpus, plus a status table per document.

Documents

Upload new PDFs/DOCX files, preview extracted text, inspect individual chunks (via a chunk dialog), and trigger reprocessing of a document if the chunking/embedding needs to be redone.

Chat / Query Playground

The employee-facing Q&A screen — ask a free-text question (with a few sample questions provided, e.g. "What is the sick leave policy for employees?"), configure how many chunks to retrieve (`top_k`), and view the answer alongside:

- A **citation card** per source chunk used (document, page, section)
- A **chunk debug card** showing vector/text/rerank scores for transparency into why a chunk was selected
- Local query history for the session

Deployment note: the Angular client is deployed live on Vercel, but its production environment config still points the API base URL at `http://127.0.0.1:8000` (localhost). In practice this means the hosted frontend only works fully when the FastAPI backend is also running locally alongside it — the backend itself is not yet deployed to a public cloud endpoint. This is worth flagging as a near-term deployment task (see Section 13).

11. Evaluation Framework

The project includes a genuine automated evaluation harness, not just manual spot-checking — this is a strong signal of engineering maturity for an early-stage project.

Test set

`eval/eval_cases.json` contains **36 hand-written test questions** tagged with tenant ID, whether the question is expected to be answerable, which document types should be cited, and which keywords the answer should contain.

Scoring script

`scripts/run_eval.py` calls `/query/ask` for every test case and scores four dimensions:

Metric	What it checks
Answerability accuracy	Did the system answer when it should, and abstain when it shouldn't?
Doc-type match	Did the citations come from the expected policy category?
Keyword coverage	Does the answer text contain the expected key terms?
Tenant isolation	Would citations ever leak across tenants? (currently unenforced — see Section 12)

Results are written to both JSON and CSV (`eval/results/`), plus a summary object with pass rates per metric — giving a repeatable regression check whenever the retrieval or prompting logic changes.

12. Current Status & Known Gaps

An honest, code-verified snapshot of what's working versus what's still in progress:

WORKING End-to-end core pipeline

- Upload → classify → chunk → embed → store, fully functional
- Hybrid retrieval (vector + text) with RRF merge and cross-encoder reranking
- Guardrailed, citation-backed answer generation with abstention behavior
- Dual LLM provider support (OpenAI / Groq) via a single config flag
- Automated evaluation harness with 36 real test cases
- Working Angular frontend covering upload, monitoring, and Q&A

IN PROGRESS Multi-tenancy

- A `/tenants` API route exists in the codebase, but it imports a model and repository module (`models/tenant.py`, `db/repositories/tenant_repo.py`) that don't currently exist in the repo, and the route is **not registered** in the main FastAPI app — so it isn't reachable yet.
- The evaluation set already has `tenant_id` fields and a tenant-isolation check, but since citations don't yet carry a tenant identifier, that check currently reports "not checked" rather than pass/fail.
- The frontend already has a "department/tenant selector" component and service scaffolded, ahead of the backend catching up.

SMALL CLEANUP ITEMS

- `/health` is defined twice — once directly in `main.py` (with a Mongo ping) and once in `api/routes/health.py` (a static OK) — the two aren't currently conflicting only because FastAPI registers the directly-defined route first, but it's worth consolidating into one.
- The `page_section_recursive` and `section_recursive` chunking strategies are mapped separately by document type, but currently run identical logic — the intended distinction (e.g. treating handbooks differently) hasn't been implemented yet.
- Backend is not deployed publicly — it currently needs to be run locally even though the frontend is hosted on Vercel.

13. Suggested Next Steps

Possible priorities for the next phase of work, roughly in order of impact:

1. **Deploy the backend** (e.g. Render, Railway, Fly.io, or a small cloud VM) and point the production frontend config at it, so the live demo is fully self-contained.
2. **Finish multi-tenancy** — add the missing tenant model/repository, register the router, and start tagging chunks/citations with `tenant_id` so retrieval can be scoped per department or client.
3. **Expand the eval set** beyond 36 cases and start tracking pass-rate trends over time as prompts/models change.
4. **Add authentication** for the upload and admin endpoints before this is exposed beyond internal testing.
5. **Differentiate the two chunking strategies** that currently behave identically, or simplify the code by merging them.

14. Appendix — Full Backend Dependency List

From `server/requirements.txt` (Python 3.12.4 runtime):

```
fastapi==0.139.0      uvicorn==0.50.2      pydantic==2.13.4
pymongo==4.17.0      langchain-core==1.4.9 langchain-openai==1.3.5
langchain-text-splitters==1.1.2 openai==2.45.0      groq==1.5.0
sentence-transformers==5.6.1      torch==2.13.0      transformers==5.14.1
pymupdf==1.28.0      pypdf==6.14.2      docx2txt==0.9
tiktoken==0.13.0      scikit-learn==1.9.0      numpy==2.5.1
python-dotenv==1.2.2      python-multipart==0.0.32 requests==2.34.2
```

Frontend (`frontend/client/package.json`): Angular 21.2, Angular Material 21.2, Angular CDK 21.2, AG Grid 36.0, RxJS 7.8, TypeScript 5.9.

Generated as a technical overview for internal review — reflects the state of the codebase as uploaded on July 30, 2026.